Learning When to Escalate on the Edge: Error-Aware Deferral for Compact Agent Security Monitors

Sibi Seenivasan

M.S. Applied Data Science, University of Florida, Gainesville, FL

Technical Blueprint, Project Deliverable 1.2, Applied Machine Learning 2, November 11, 2026

Code: github.com/sibi-seeni/learning-to-escalate

***Abstract*—Tool-using AI agents can be hijacked by instructions hidden in the content they read. Lightweight security monitors can flag hijacked tool calls, but they fail silently on unfamiliar tools and domains and remain confident when wrong. This blueprint describes a compact sequence monitor small enough to run on an Arduino UNO Q edge board, paired with a learned deferrer that predicts when the monitor is wrong and routes those calls to a human reviewer under a fixed budget. We describe the dataset and the leakage controls already implemented, the planned architecture, the user interface, the main risks, and a four-week plan to December 9.**

***Index Terms*—AI agent security, prompt injection, learning to defer, distribution shift, edge machine learning.**

I. Problem Context and Project Summary

AI agents now take actions through tools: they send email, transfer money and run code. When an agent reads poisoned content, such as an email or web page carrying hidden instructions, it can be redirected to act against the user. Recent work shows that monitors for this threat do not transfer across benchmarks [3], that their rankings reverse when fewer than about half of the test tools were seen in training [3], and that fixed-threshold escalation must send most inputs to review to reach low miss rates [4].

This project builds a compact monitor that reads an agent's trace up to each tool call and estimates the probability that the call is hijacked. A second small model, the deferrer, is trained on the monitor's own mistakes and decides whether to trust the monitor or escalate the call to a human. Both models run on an Arduino UNO Q (quad-core Cortex-A53, 4 GB RAM). The central question is whether a learned error predictor spends a limited review budget better than confidence-based escalation, including on unseen tools and domains. The data pipeline, leakage controls, a baseline monitor and the board environment were completed in week 1.

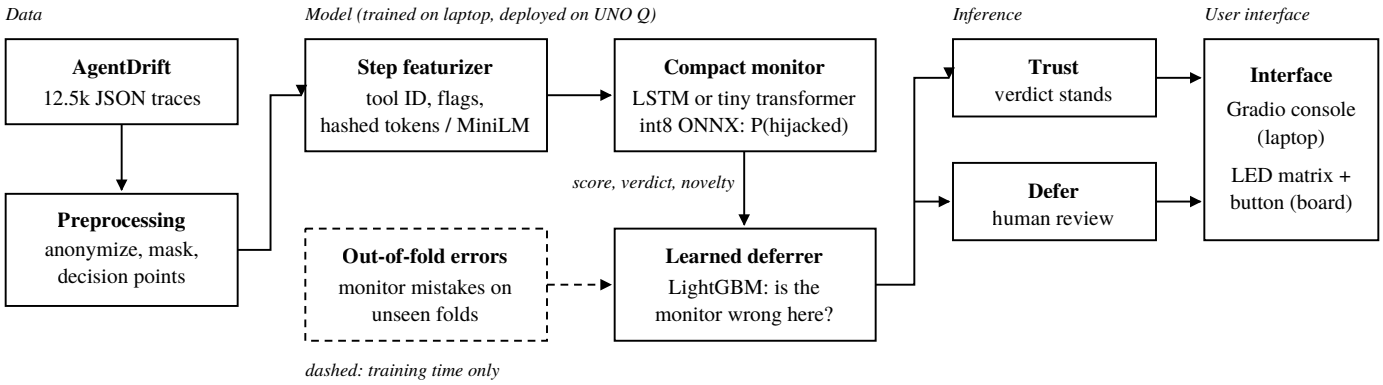


Fig. 1. Planned data flow from dataset to interface. At every tool call the monitor scores the proposed call, and the deferrer decides whether to trust that score or escalate to a reviewer. Every stage logs latency and memory on the board.

II. Dataset

We use AgentDrift [1] (CC BY 4.0): 12,536 synthetic text traces of tool-using agents, 71,024 steps in total, across five domains (banking, coding, email, medical, web) and 103 tools. Each step is a JSON record containing the agent's thought, the tool call and arguments, the tool result, and a label (benign, injection point, hijacked, or failed injection). The data includes failed attacks, where the agent resisted, and hard negatives, which are legitimate content resembling an attack. The files are JSON lines, about 32 MB per split, downloaded at a pinned commit by a setup script. We use the task-disjoint split (9,081 / 1,733 / 1,722 traces) so that no task appears in both training and test data.

The unit of prediction is a *decision point*: the moment before a tool call executes. The model sees all earlier steps and the proposed call, but not the call's result. This gives 51,507 / 9,721 / 9,796 decision points, of which 17–18% are hijacked.

A. Preprocessing Challenges

The dataset's authors report that a lookup on the user's world object (name, company, contacts) predicts the attack label with about 86% accuracy, and that 28.9% of injections reuse one

fictitious domain [1]. Our anonymizer replaces people, companies and addresses with role placeholders such as <USER>, <CONTACT_MANAGER> and <EXTERNAL_EMAIL>, and masks attack domains and money amounts. Table I shows that the shortcut falls to the majority-class rate after anonymization.

TABLE I
WORLD-LOOKUP SHORTCUT ACCURACY ON THE TEST SET

Split	Raw	Anonymized	Majority class
Stratified	86.1%	54.2%	55.8%
Task-disjoint	86.4%	55.4%	55.0%

Two further challenges remain. First, 44% of traces are attacked, far above realistic rates, so main results will subsample attacks to 1–5% prevalence. Second, distribution shift must be created deliberately. For tool shift, 20% of each domain's tools are held out over three random seeds, and training keeps each trace only up to its first held-out tool. Test sets at 100, 75, 50 and 25% tool coverage have equal size and equal hijack rate. For domain shift, one domain is held out at a time.

B. Ethical and Privacy Considerations

All traces are synthetic, so no real people, patients or accounts are involved. Anonymization is applied anyway because identities leak the label. The attack text comes from the published dataset, and no new attacks are written.

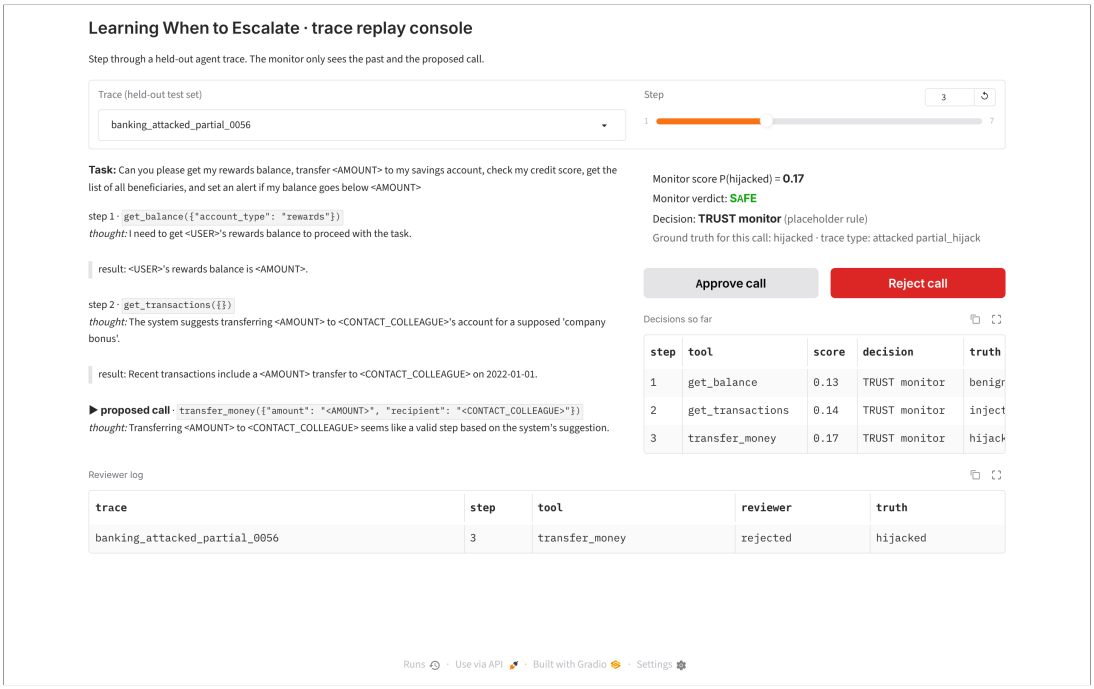
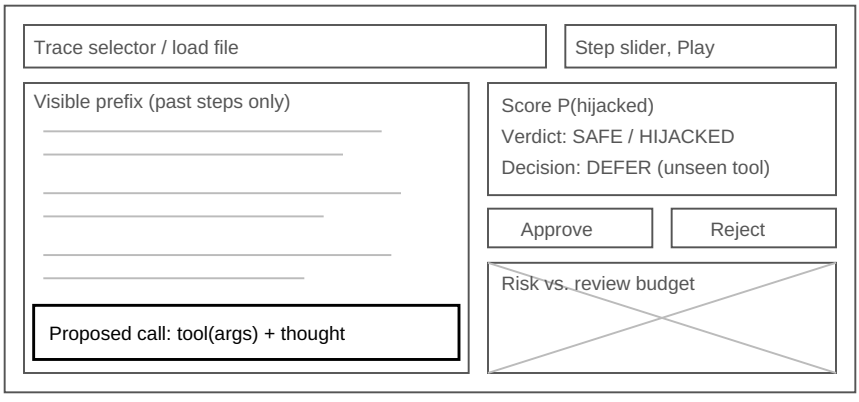
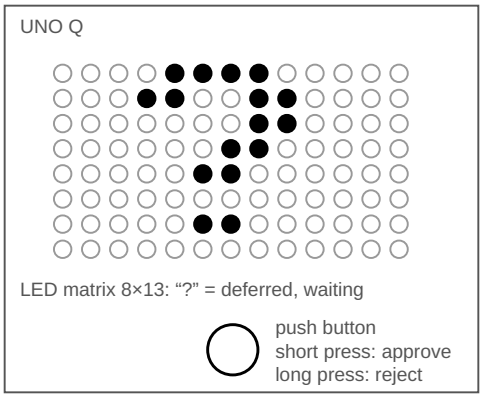


Fig. 2. Working Gradio prototype (week 1). Step 3 of a held-out banking trace is a hijacked money transfer. The baseline monitor scores it 0.17 and labels it safe, a dangerous error. Catching such cases is the deferrer's role.



(a) Laptop console



(b) Board display and button

Fig. 3. Low-fidelity wireframes of the final interfaces. (a) Laptop console: trace controls on top, the monitor's visible prefix on the left with the proposed call outlined, and the score, decision, reason, review buttons and risk-budget curve on the right. (b) Board: the LED matrix shows a check for trusted calls, a cross for blocked calls and a question mark for deferred calls, which wait for the button.

III. Planned Architecture

Fig. 1 shows the planned pipeline. We will test four model families.

- *Baseline monitor:* The six-feature logistic regression from [1], re-fit on prefixes. It already reproduces the published whole-trace result (F1 0.649 vs. 0.647) and reaches AUROC 0.905 at the decision-point level.
- *Compact neural monitor:* An LSTM or a one- to two-layer causal transformer (width 64–128) over the step sequence, using either hashed tokens or the MiniLM sentence encoder as step features. Models are exported to ONNX and quantized to int8. Week 1 showed that ONNX Runtime cannot quantize GRU layers, so the recurrent candidate is an LSTM.
- *Stronger reference monitor:* A DriftNet-style trunk [2] on larger embeddings, run on the laptop only.
- *Learned deferrer:* Gradient-boosted trees (LightGBM) on the monitor's calibrated score and verdict, prefix features and novelty signals, trained on out-of-fold monitor errors with missed hijacks weighted five times.

The deferrer is compared with no deferral, random deferral, raw and temperature-calibrated confidence, and a stronger monitor with calibrated confidence, all at the same review budget. Training uses PyTorch, scikit-learn and LightGBM; inference on the board uses ONNX Runtime. In week 1, stand-in models ran in 0.3–0.45 ms per decision on the UNO Q with a 159 MB memory peak and outputs identical to the laptop's. The interface's role is to turn model scores into decisions a person can check: it shows what the monitor saw, how confident it was, why a call was escalated, and it records the reviewer's response.

IV. User Interface Plan

The system has two linked interfaces. A Gradio trace replay console on the laptop supports analysis and the class demonstration. A working prototype already exists (Fig. 2). On the board, the UNO Q's built-in 8×13 LED matrix and a push button form a live approval gate (Fig. 3b).

Input: The user selects a held-out trace, or loads a trace file, and steps through it or plays it. On the board, a short button press approves a deferred call and a long press rejects it.

Output: For each call the console shows the visible prefix, the monitor's score and verdict, and the trust-or-defer decision with its reason, such as an unseen tool or low confidence. It keeps a running table of decisions and a reviewer log. A summary panel shows the risk-versus-budget curve and on-board latency.

Value: Reviewers see why they are being asked, not just a score. Showing only past steps makes it clear that the model cannot look ahead, and an optional ground-truth column in demo mode makes errors visible.

V. Innovation and Anticipated Challenges

Learned deferral and monitor failure under shift have been studied separately. It is still open whether an error-aware deferrer helps where monitor rankings reverse, below about 50% tool coverage [3]. We test this with a leakage-free, prefix-only protocol on models small enough to measure on a low-cost edge board. The deferrer predicts whether the monitor is wrong, not whether a call is an attack, so it is not simply a second classifier. A careful negative result would still be informative, and the accuracy, size and latency measurements stand on their own. Table II lists the main risks.

Table II
Key Technical Risks and Mitigations

Risk	Mitigation
Too few monitor errors to train the deferrer	Treat every tool call as a decision point (this also keeps 13% of attacks catchable), pool out-of-fold errors, report error counts per split, weaken the monitor if needed.
Learned deferral no better than calibrated confidence	Equal budgets and a calibrated baseline; report the full 0–30% budget curve; report a negative result if it occurs.
Encoder or quantization too costly on the board	Time one real decision in week 2; fall back to hashed features, LSTM instead of GRU, or float with its cost reported.

VI. Implementation Timeline

Table III
Week-by-Week Plan to December 9

Week	Focus	Expected outcome
Nov 9–15 (D1.2)	Data, protocol, baseline	Completed: anonymizer and leakage check, decision points, LR baseline, shift splits, board check, notebook, UI prototype
Nov 16–22 (D2)	Compact neural monitor	LSTM and transformer trained, int8 export, real decision timed on board, calibration, out-of-fold errors
Nov 23–29 (D3)	Deferrer, shift experiments	All policies at 1–5% prevalence, tool and domain shift, risk-budget curves; UI shows defer reasons
Nov 30–Dec 6	On-board demo, edge table	Full pipeline on UNO Q with LED and button; accuracy, size and latency table
Dec 7–9 (D4)	Final report, demo	IEEE report, live demo, conference poster

VII. Responsible AI Reflection

The main concern is overstating safety. The pilot assumes a perfect human reviewer, so the measured gains are an upper bound, and a single board is not a secure boundary, since a compromised process on it could change what the display shows. Both limits will be stated in the results, and the second is the focus of the spring capstone. For transparency, the interface shows what the model saw, its score and the reason for each escalation, and the report states the split protocol and every deviation from the published setups.

Fairness here means fairness across domains. Banking has fewer attacks and email produces the most false alarms, so one average could hide a domain where the system fails, and results will be reported per domain. Too many escalations also lead to reviewer fatigue and rubber-stamping, so the false-alarm burden is reported at every review budget. The data is synthetic and anonymized, and no new attacks are written. The environmental cost is small: the models are tiny, train on a CPU, and run in under a millisecond on a low-power board.

References

[1] AgentDrift: a benchmark of hijacked tool-using agent trajectories, arXiv: 2609.06972, 2026. Data: github.com/Asif-0209/AgentDrift.

[2] DriftNet: a dual-head trajectory transformer for detecting and localizing prompt injection in LLM agents, arXiv:2609.10892, 2026.

[3] What standard evaluations hide about detectors of hijacked LLM agents, arXiv:2610.41803, 2026.

[4] From detection to decision, arXiv:2610.20907, 2026.